Milestone 4: Secure Authentication and Authorization System

Secure Delivery Defect and Station Performance Dashboard

Omar Soto Palos

SWE-550: Software Engineering and Security Principles

Instructor: Shane Carroll

October 25, 2026

Project Overview

The Secure Delivery Defect and Station Performance Dashboard is a Flask application built with Python and SQLite. It uses fictional delivery records rather than confidential Amazon data. Milestone 4 focuses on verifying user identities, assigning the right permissions, and protecting authenticated sessions. The goal is to make sure users can access only the pages and functions needed for their roles.

Repository: https://github.com/Ramo3695/Secure-Delivery-Defect-Dashboard/tree/main

Application code: https://github.com/Ramo3695/Secure-Delivery-Defect-Dashboard/blob/main/app.py

Project documentation: https://github.com/Ramo3695/Secure-Delivery-Defect-Dashboard/tree/main/docs

Milestone 4 screencast: Attached separately.

Authentication and Account Creation

User accounts are stored in SQLite. Each account has a unique username, a password hash, and an assigned role. The create user() function checks that usernames are between 3 and 50 characters, passwords contain at least 12 characters, and roles match the approved list. Python tool handles password hashing and verification, while Flask manages login sessions.

The application includes an administrator-only Create User Account page at /admin/create-user. The route requires an authenticated administrator and uses a CSRF token to protect the form submission. The first administrator account was created locally as a setup step; afterward, an administrator can create accounts through the application.

I signed in as testadmin and created a test account through the administrator page. My application displayed “User account created successfully.” I also tested valid and incorrect login information, logging out, and what happened when someone tried to reopen the dashboard without signing in. The screenshots below show the results I collected.

Role-Based Authorization Design

The dashboard uses three roles: admin, manager, and team_member. Administrators can access the dashboard, manage delivery records, open the administrator panel, and create user accounts. Managers can view the dashboard and manage delivery records. Team members can view the dashboard but cannot manage records or access administrator functions. The login_required and role_required decorators enforce these permissions on the server by checking the user account in SQLite.

Access Control Matrix

Resource / route	**Admin**	**Manager**	**Team member**	**Logged out**
Dashboard /dashboard	Allow	Allow	Allow	Deny/redirect
Manage records /manage-records	Allow	Allow	Deny (403)	Deny/redirect
Admin panel /admin	Allow	Deny (403)	Deny (403)	Deny/redirect
Create account /admin/create user	Allow	Deny (403)	Deny (403)	Deny/redirect

Logout uses a POST request protected by a CSRF token. When a user logs out, Flask clears the session. The login page remains available without authentication.

Access Control Testing

ID	Account / resource	Expected	Actual / evidence
AC-01	staff/admin/create user	403 denied	403 screenshot, Oct. 27; passed
AC-02	testmanager/admin/create user	403 denied	403 screenshot, Oct. 27; passed
AC-03	testmanager/admin	403 denied	403 screenshot, manager /admin; passed (Figure 13)
AC-04	staff/manage records	403 denied	403 screenshot, staff /manage-records; passed (Figure 10)
AC-05	staff/admin	403 denied	403 screenshot, staff /admin; passed (Figure 11)
AC-06	testadmin/admin/create-user	Allowed	Account created; success screenshot
AC-07	Logged out/dashboard	Redirect to login	Redirect to login after logout; passed (Figure

			8)

I tested the restricted pages by entering their URLs directly in Chrome. I wanted to confirm that users could not get around their permissions just by knowing a page address. I compared what I expected with what actually happened, and I saved screenshots showing the signed in role and the blocked pages. A denial screenshot by itself does not show the account, so I included the matching dashboard views where possible.

Credential and Session Protection

Control	**Implementation and risk addressed**
Password storage	Python tool hashes passwords before storage and verifies them during login. The database does not store plaintext passwords.
Session hijacking	HTTPOnly limits JavaScript access to the session cookie, and SameSite=Lax reduces some cross site cookie exposure. HTTPS and Secure cookies are still needed for deployment.
Session fixation	After valid credentials are accepted, session.clear() removes previous session values before the application establishes the authenticated session. Flask uses signed client side session cookies rather than server side

	session IDs.
Session expiration	The application checks the time since the previous request and clears an inactive session. For the Milestone 4 demonstration, the configured timeout is temporarily one minute; the intended standard setting is 15 minutes.
Logout	Logout clears the authenticated Flask session and redirects the user to the login page. Direct attempts to reopen protected pages after logout are redirected to login.
Session misuse	Role checks read current account permissions from SQLite. CSRF protection rejects forms with missing tokens, and IP-based login limits reduce repeated credential attempts.

The application is currently used locally over HTTP, so SESSION_COOKIE_SECURE is set to False for development. HTTPOnly limits JavaScript access to the session cookie, and SameSite=Lax provides some protection against cross site requests. A real deployment would require HTTPS, Secure cookies, persistent rate limit storage, and stronger audit logging.

STRIDE Threat Mitigation

Threat	**Implemented mitigation**	**How it reduces risk**
Spoofing	Passwords are hashed and checked with python tool. Flask-Limiter allows five login submissions per 15 minutes per IP address.	This reduces repeated password guessing and requires valid credentials for authentication.
Elevation of privilege	Backend authorization checks the user's role in SQLite before granting access to restricted routes, including account creation.	A staff member or manager cannot obtain administrator access simply by entering the restricted URL.
Information disclosure	The application uses role restricted pages, generic error responses, and Flask debug mode set to False.	These controls help prevent unauthorized viewing of protected features and avoid exposing detailed error information to users.

Security Testing Screenshots

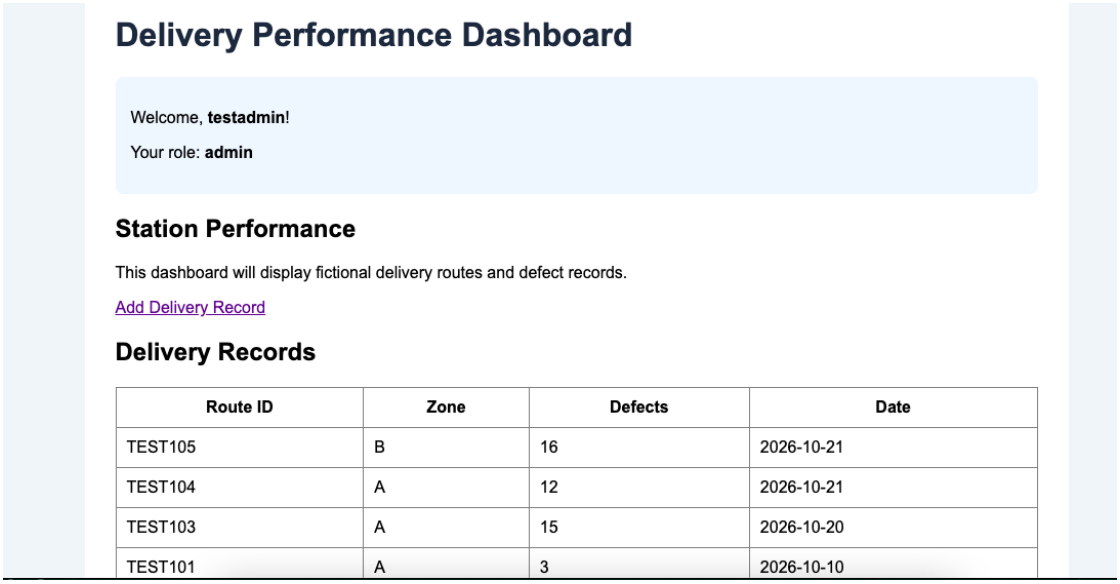


Figure 1. My administrator dashboard showing the admin role.

When I signed in as testadmin, the dashboard showed my username and the admin role.

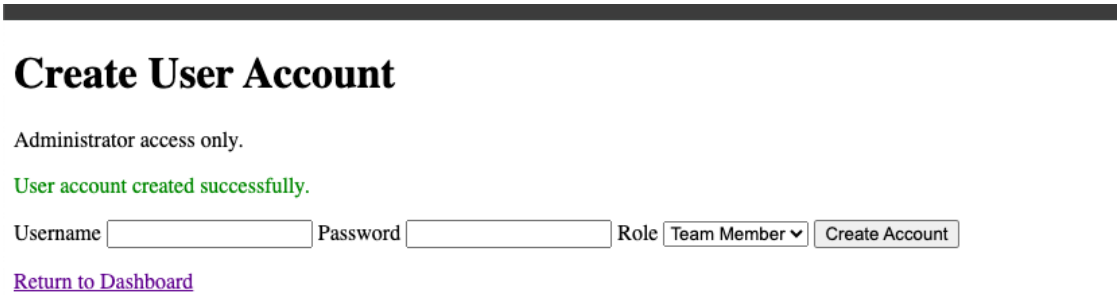


Figure 2. The success message after I created a test account.

After I submitted the account creation form, the application confirmed that the new user was created.

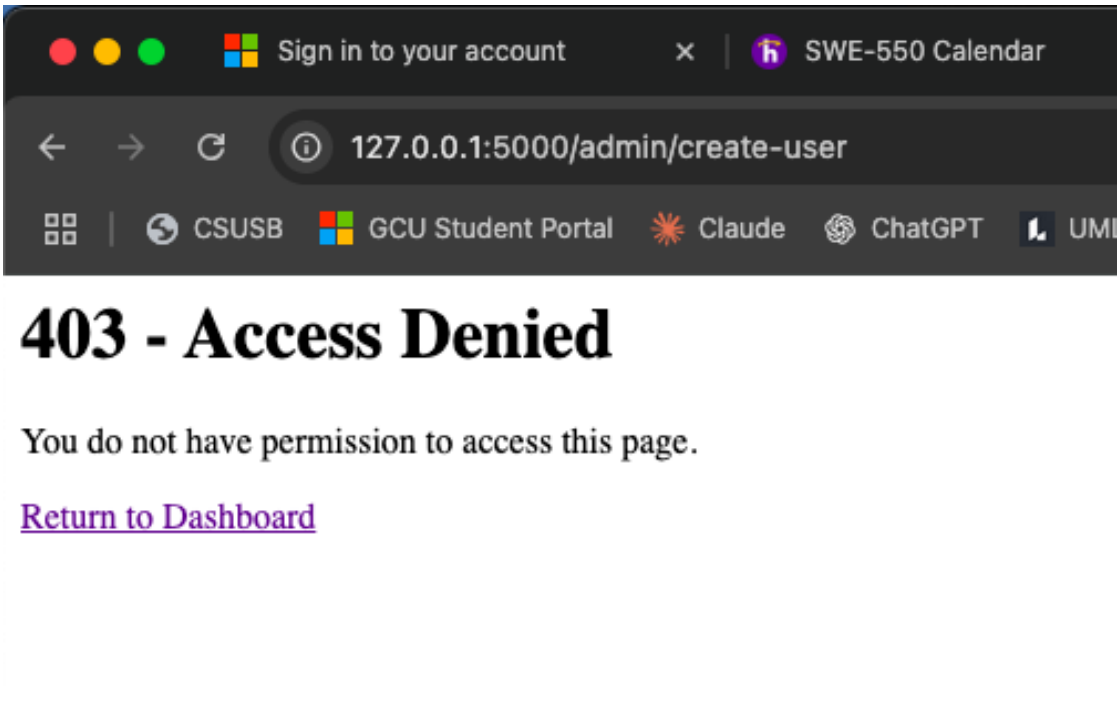


Figure 3. Staff account blocked from the Create User Account page.

I entered /admin/create-user directly while testing the staff account, and the application denied access with a 403 error.

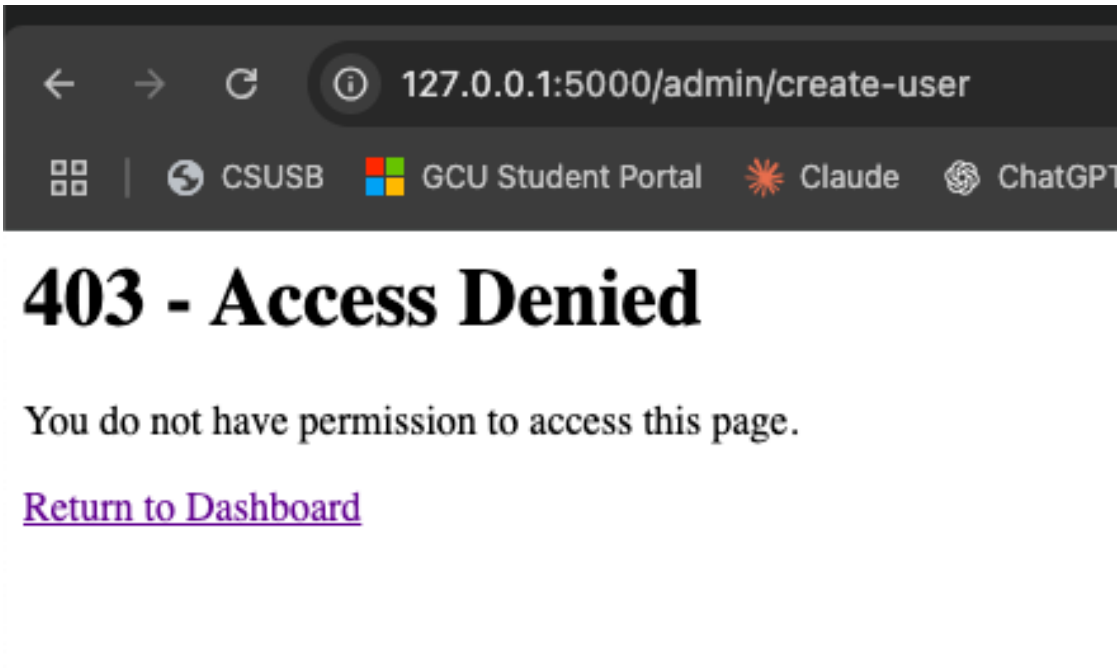


Figure 4. Manager account blocked from the Create User Account page.

I tried opening: /admin/create-user as testmanager, and the application returned a 403 error.

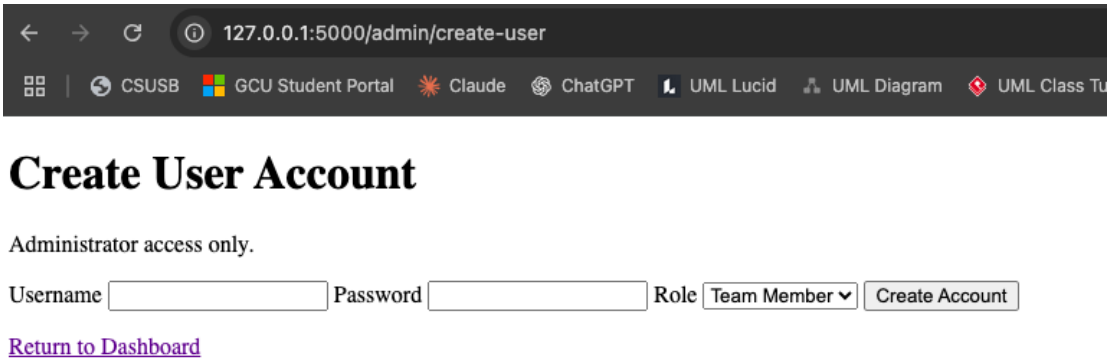


Figure 5. The account creation form available to an administrator.

This is the form I created for administrators. It lets an authorized admin enter a username, password, and role. This screenshot shows the form, while Figure 2 shows a successful submission.

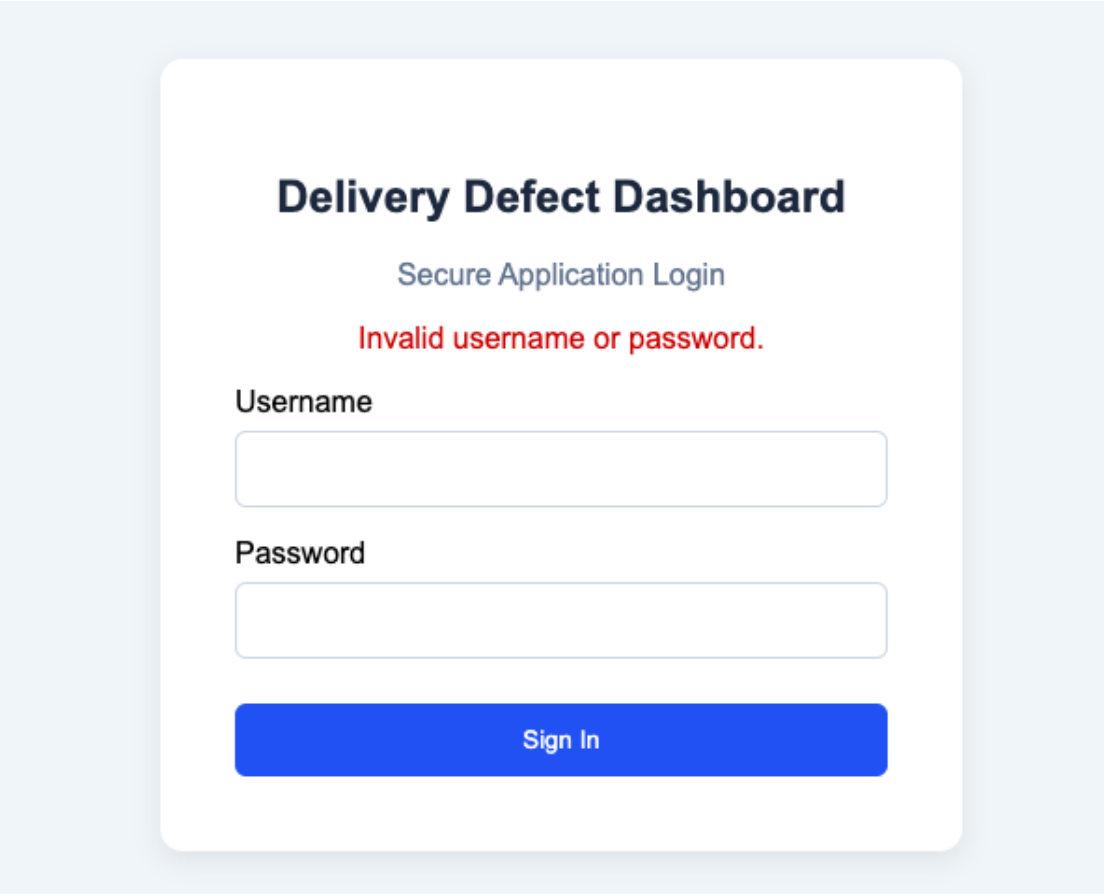


Figure 6. Login screen rejecting an incorrect username or password.

I entered incorrect login information, and the application showed “Invalid username or password” instead of allowing access to the dashboard.

The additional screenshots document successful login, logout, role assignments, blocked direct URL access, and the inactivity timeout. For the demonstration, the timeout was temporarily reduced to one minute. A login page alone does not establish why a session ended, so the timeout setting, and the sequence of actions will be demonstrated in the screen cast video.

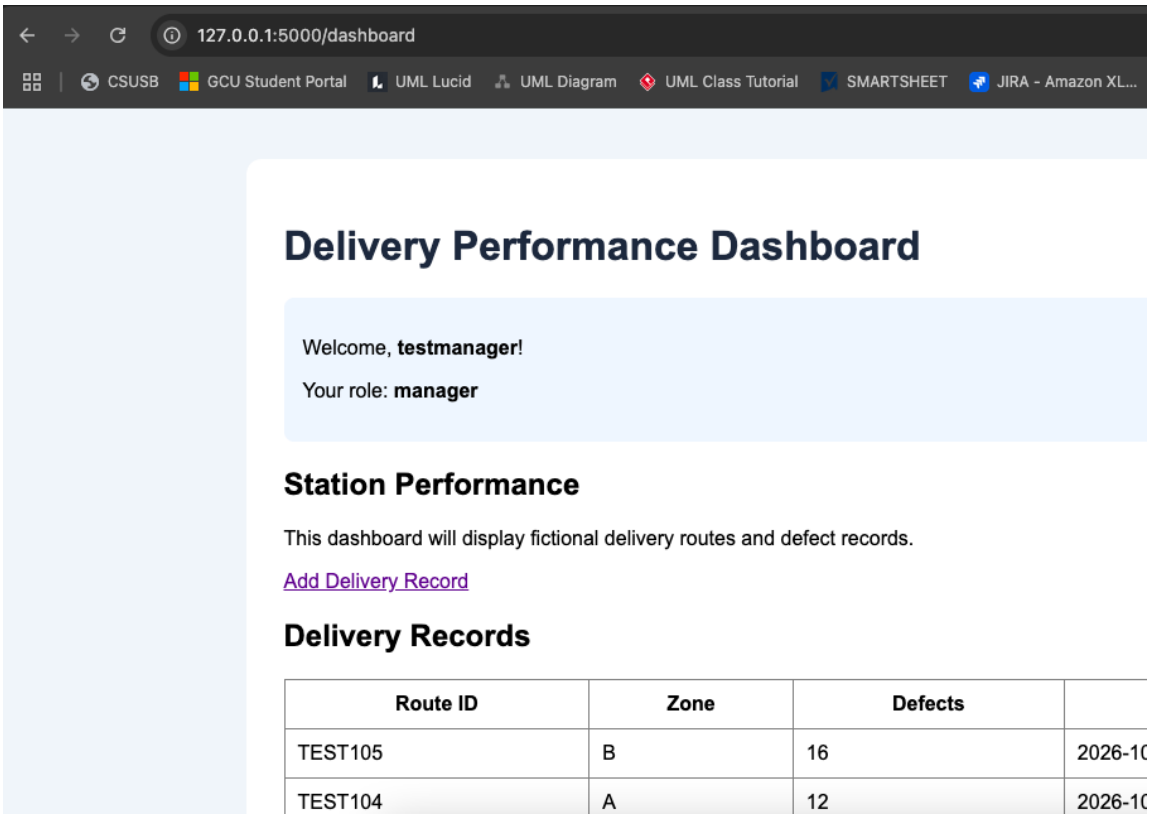


Figure 7. Successful login to the manager dashboard.

I signed in as testmanager. The dashboard showed my username, my manager role, and the delivery records, so I could confirm the login worked.

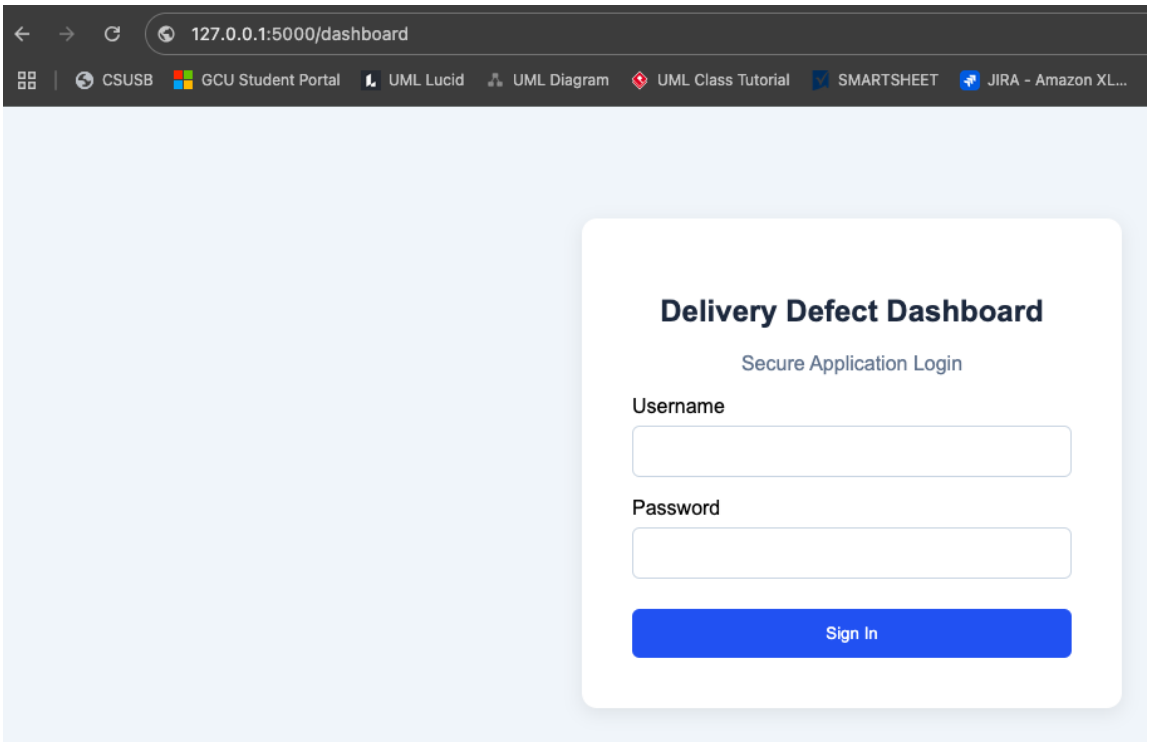


Figure 8. Dashboard access blocked after logout.

After logging out, I entered /dashboard directly. The application sent me back to the login page instead of showing the records.

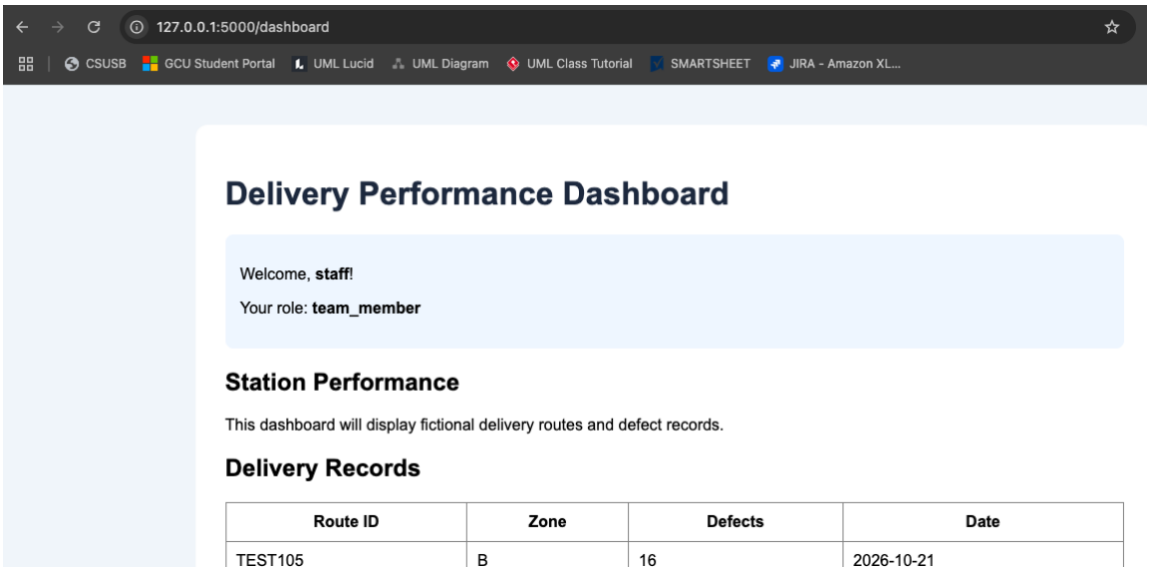


Figure 9. Staff account shows team member role.

I logged in as staff, and the dashboard showed the team_member role. This account can look at the records but should not be able to manage them.

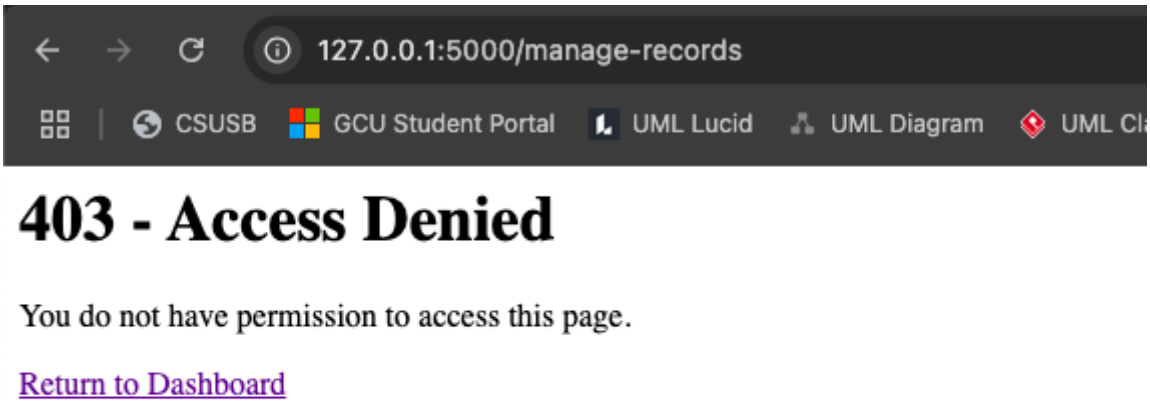


Figure 10. Staff denied access to Manage Records.

While using my staff account, I entered /manage-records directly. The application showed a 403 Access Denied message, which matched the permission I assigned.

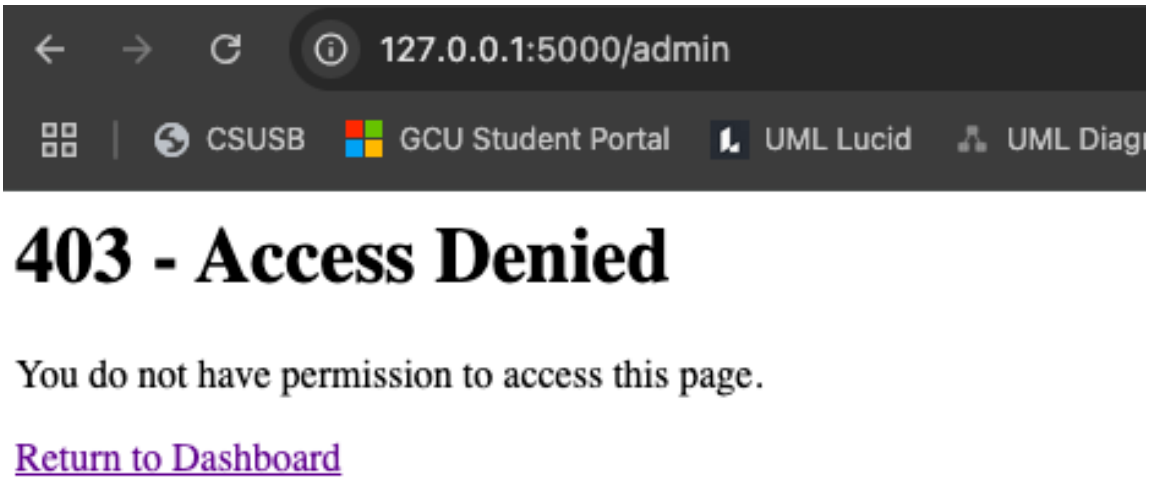


Figure 11. Staff denied access to the administrator panel.

I tried opening /admin while signed in as staff. My application returned 403 Access Denied because team members do not have administrator permissions.

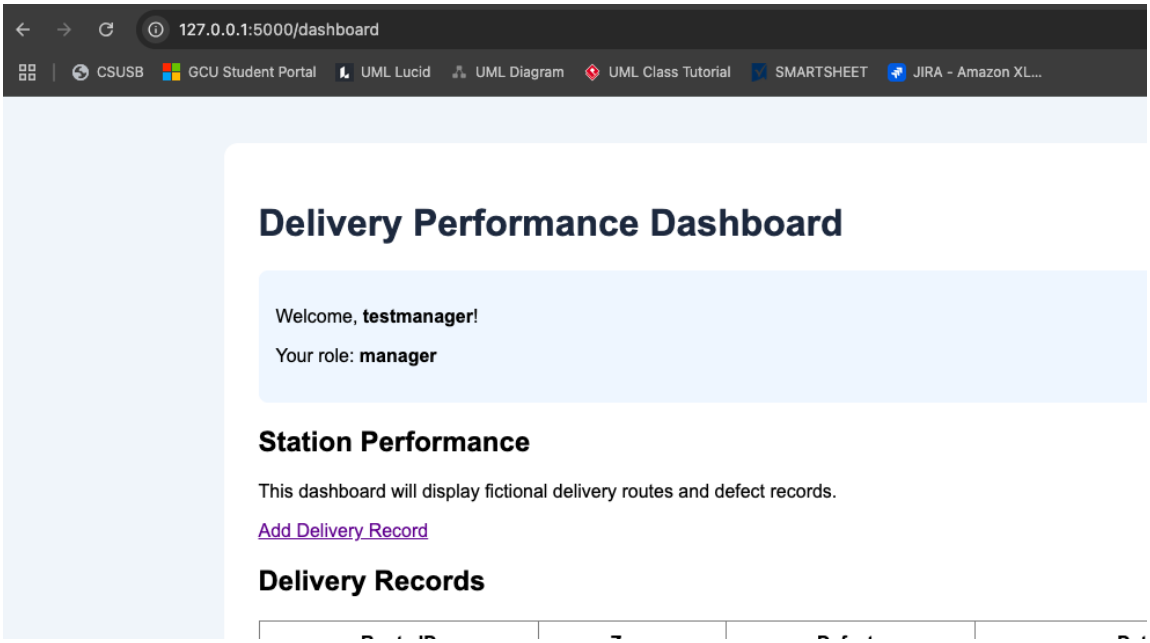


Figure 12. Manager account shows assigned role.

I signed in as testmanager again. The dashboard displayed the manager role and the link to add delivery records.

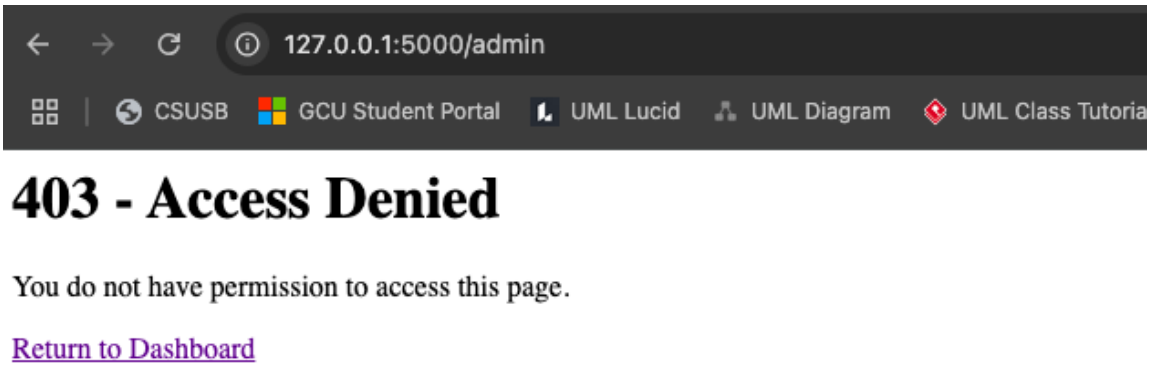


Figure 13. Manager denied access to administrator panel.

I typed /admin into Chrome while signed in as testmanager. The application blocked the page with 403 Access Denied, as expected.

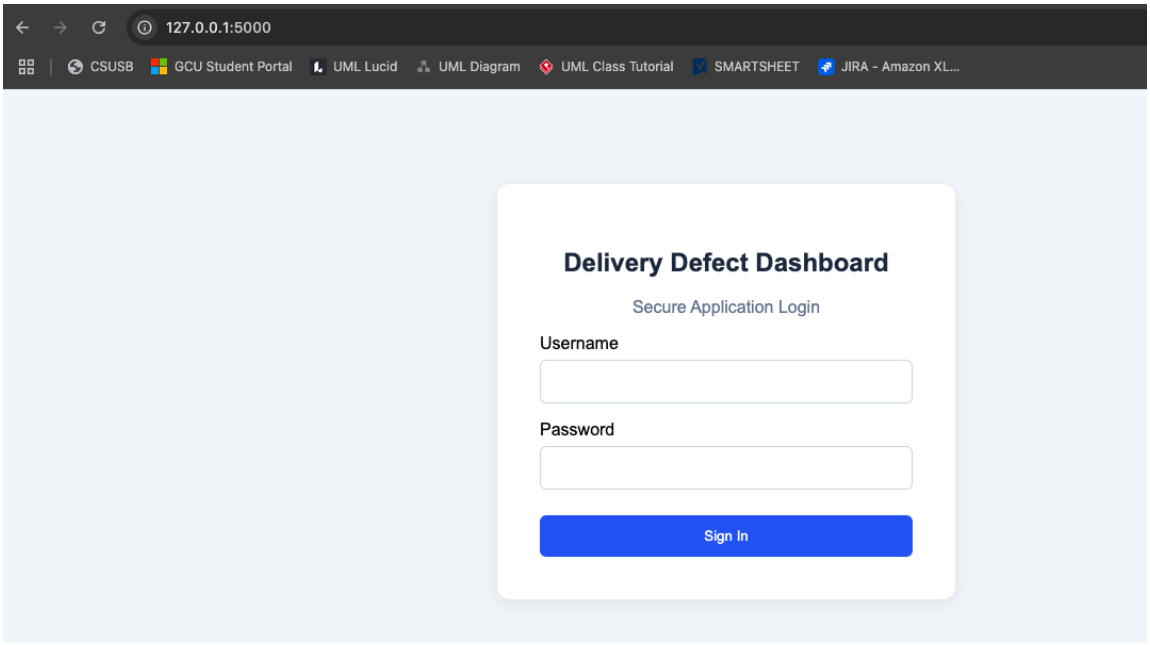


Figure 14. Login required after session inactivity.

I left the manager dashboard inactive longer than the one-minute demonstration timeout. When I tried to return to a protected page, the application redirected me to login. I will show the timeout setting during the video so the reason for this redirect is clear.